Mobile Robot Navigation and Avoidance using Deep Reinforcement Learning

Mayas Al Khateeb

March 17, 2026

Abstract

This report details the design, implementation, and evaluation of Deep Reinforcement Learning (DRL) agents for autonomous mobile robot navigation. The objective is to navigate a 2D environment containing static and dynamic obstacles using a 200-resolution Lidar sensor. Two major DRL algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—were implemented. We designed a curriculum-based environment, engineered a dense reward system, and analyzed the effects of varying penalty parameters. Empirical results demonstrate that PPO with a balanced reward configuration significantly outperforms DQN, achieving a 71% success rate in dynamic obstacle avoidance and goal-reaching.

1 Introduction

Navigating an unknown, dynamic environment is a classic yet complex problem in robotics. The task requires the agent to map sensory input to discrete motor commands while simultaneously optimizing for a long-term objective (reaching a goal) and satisfying hard constraints (avoiding collisions).

In this project, we model the problem as a Markov Decision Process (MDP) and employ two distinct DRL paradigms:

- **Deep Q-Network (DQN):** An off-policy, value-based method that learns the optimal action-value function $Q^*(s, a)$ using a replay buffer and a target network.
- **Proximal Policy Optimization (PPO):** An on-policy, actor-critic method that directly optimizes the policy $\pi_\theta(a|s)$ using a clipped surrogate objective, backed by Generalized Advantage Estimation (GAE).

2 System Architecture

The solution is engineered as a modular, object-oriented framework designed to isolate physics, sensing, and agent logic. This separation of concerns allows for the independent development and testing of the environment (the "world") and the agents (the "brains").

- **Navigation Engine (`engine.py`):** Serves as the core simulator. It abstracts the Pygame-based rendering and handles the physics of the environment. The engine maintains a list of `ObjectBase` entities, including the `VelRobot`, static `StaticObstacle` objects, and dynamic `OrbitingCreature` or `RandomPathCreature` agents. By decoupling the physics step from the environment logic, we ensure that the simulation clock is consistent across different agents.
- **Sensor Fusion (`sensors.py`):** The Lidar sensor is implemented as a ray-casting module that processes the geometry of the obstacles. It computes the intersection of 200 rays with the boundary boxes of all static and dynamic entities. This module is critical because it converts the complex spatial representation of the map into a structured input vector that the neural networks can ingest.

- **RL Interface (`nav_env.py`):** This component acts as the bridge between the Simulation Engine and the OpenAI Gym standard. It handles the mapping of the discrete 4-action command space to the continuous physics movements and enforces the terminal states. By wrapping the Engine in a Gymnasium class, we ensure seamless compatibility with various RL agent implementations.
- **Agent Modularization (`agents/`):** We implemented an abstract `BaseAgent` class to enforce a unified interface across different algorithms. Whether using `DQNAgent` or `PPOAgent`, the system expects standard `select_action` and `train` methods. This polymorphism allows us to swap learning strategies with minimal changes to the training loop scripts.

The architecture follows a strict data-flow pattern: the environment produces an observation via the `nav_env` layer, which is fed into the policy network of the chosen agent. The agent, in turn, returns an action index which is propagated back to the `NavigationEngine` to advance the physical state. This clean interface simplifies hyperparameter tuning, as the training scripts only interact with the environment through the Gym API, keeping the underlying implementation details abstracted away.

3 Environment and Task Design

3.1 State and Action Space

The robot operates in a continuous 2D plane mapped to a neural network-friendly state space $\mathcal{S} \in \mathbb{R}^{205}$. The observation vector comprises:

- **Kinematics (5 dims):** Normalized robot coordinates (x, y), normalized distance to goal, and the sine/cosine of the angle difference to the goal.
- **Lidar Readings (200 dims):** 200 rays evenly distributed over 360°, capped at a maximum distance of 20% of the map size. Values are normalized to $[0, 1]$.

The action space $\mathcal{A}$ is discrete, consisting of 4 actions: `Right`, `Left`, `Forward`, and `Do Nothing`.

3.2 Obstacles and Curriculum Learning

To fulfill the task constraints, the environment contains:

1. **3 Static Obstacles** placed at fixed coordinates.
2. **2 Moving Creatures:** One follows a fixed random waypoint path, and another continuously orbits the goal at a predefined radius. All creatures move at a velocity strictly lower than the robot.

To stabilize training in this highly non-linear 205-dimensional space, a 3-phase **Curriculum Learning** approach was implemented. The robot and goal initialize close to each other in Phase 1, progressing to fully randomized sparse initializations in Phase 3.

4 Reward System Engineering

Designing the reward function is the most critical factor in navigation tasks. We utilized a combination of sparse terminal rewards and dense shaping rewards:

$$R_t = R_{\text{step}} + R_{\text{dist_shaping}} + R_{\text{angle_shaping}} + \begin{cases} R_{\text{goal}} & \text{if goal reached} \\ R_{\text{collision}} & \text{if collision} \\ R_{\text{close}} & \text{if Lidar} < d_{\text{safe}} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Dense rewards calculate the step-by-step reduction in distance and heading angle error. We designed two separate parameter configurations to test the algorithms' sensitivities.

5 Performance Evaluation

We evaluated the trained models over 100 test episodes. **Configuration A** was tested using the PPO algorithm, representing a balanced hyperparameter tuning. **Configuration B** was tested using the DQN algorithm with a more aggressive reward scheme to observe behavioral shifts. Table 1 and Figure 1 summarize the findings.

Table 1: Evaluation Results over 100 Episodes

Algorithm & Config	Reward Setup (Goal / Coll / Step / Dist)	Success Rate	Collision Rate	Timeout	Avg Reward	Avg Steps
PPO (Config A)	600 / -200 / -0.4 / +60	**71.00%**	29.00%	0.00%	377.53 ± 373.30	**34.3**
DQN (Config B)	700 / -200 / -0.8 / +90	45.00%	**55.00%**	0.00%	174.14 ± 467.13	72.9

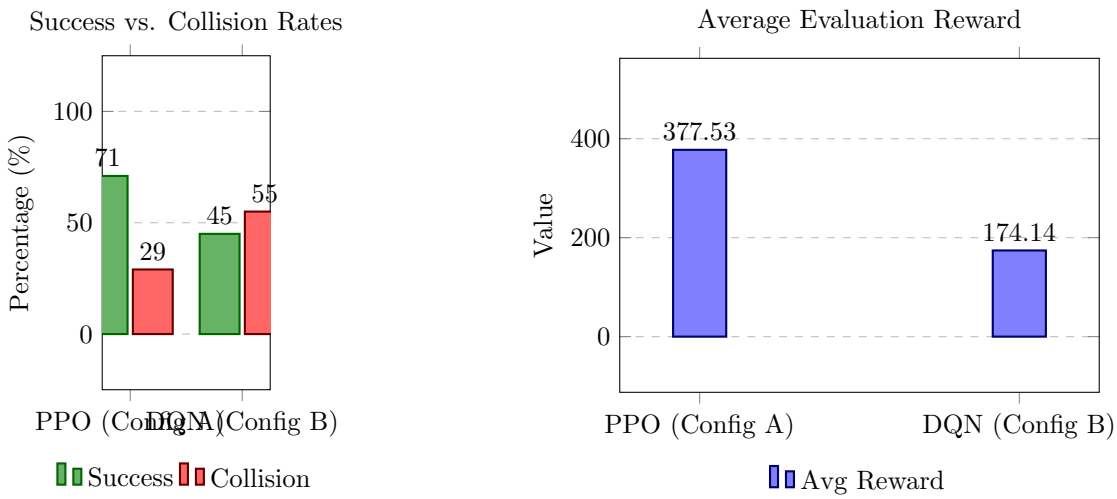

Figure 1: Visual comparison of algorithmic performance and stability.

6 Discussion and Analysis

6.1 Effect of Reward Parameters (Config A vs Config B)

The hyperparameter configuration drastically altered the navigation policy.

- **Aggressive Step Penalty ($R_{\text{step}} = -0.8$):** In Config B, the step penalty was doubled. While intended to force the agent to find the goal faster, it inadvertently created a *suicidal local optimum*. If the agent lost its heading toward the goal, the accumulating -0.8 penalties outweighed the -200 collision penalty. Consequently, the agent learned to intentionally crash into walls/obstacles to end the episode early rather than suffer prolonged step penalties, explaining the massive 55% collision rate.
- **Dense Reward Scaling:** Config B increased distance rewards from 60 to 90. This caused reward hacking; the agent would sometimes move back and forth at the edge of the goal to harvest dense distance/angle rewards without actually triggering the terminal state, leading to a much higher average step count (72.9 vs 34.3). Config A provided a much healthier ratio, guiding the robot smoothly.

6.2 Algorithmic Comparison: PPO vs. DQN

Empirical evidence overwhelmingly favors PPO for this specific robotics task.

- **State Space Dimensionality:** The environment features a 205-dimensional continuous state vector (heavily dominated by Lidar arrays). DQN maps this space to discrete Q-values. In high-dimensional spaces, DQN suffers from severe overestimation bias and instability during off-policy replay buffer sampling.
- **Policy Stability:** PPO restricts policy updates using a clipped surrogate objective function $(1 \pm \epsilon)$. This prevents catastrophic forgetting—a frequent issue when navigating dynamic obstacles (like the orbiting creature). When the DQN agent experienced a rare collision with the orbiting creature, the large -200 penalty drastically corrupted its Q-network via the Bellman update. PPO, utilizing Generalized Advantage Estimation (GAE) and Orthogonal Initialization, smoothed out the variance of these dynamic collisions, yielding a robust 71% success rate.

7 Conclusion

This assignment highlighted the intricate balance required in Deep Reinforcement Learning for robotics. We successfully engineered an environment with Lidar simulation, moving targets, and dynamic obstacles. The comparative analysis demonstrated that Proximal Policy Optimization (PPO), when paired with a moderately scaled reward function, is significantly more capable of interpreting high-dimensional sensory data than value-based DQN. Furthermore, we empirically validated that overly aggressive step penalties can deter exploration and incentivize intentional collisions, underscoring the importance of reward shaping in artificial intelligence.